# Exercise 4: Two Skills and a Secret Guard

**Session**: 4 — Skills, Hooks & Guardrails
**Duration**: 40 minutes, plus a 5-minute closing round
**Project**: The same BookStore API.

## Goal

1. Copy a ready-made `/commit` skill and run it.
2. Let Claude write a `/changelog` skill. Write its `description` yourself and test it.
3. Wire a hook that blocks a prompt with a secret key in it. Find what the hook misses, and close that gap with a rule.

## Start here

- Open a terminal in the `bookstore` folder that you cloned, and start `claude`. The skills, hook and settings you create go in its `.claude/` folder.
- The file `.env` holds a fake API key. In this exercise, treat it as a secret.
- Training mode is on. Copy tagged prompts exactly, tag included. Stuck? Type *just tell me* in the chat.
- Never edit `CLAUDE.md`.
- The trainer says when to start each next task. Start it, even if you are not done.

## Tasks

### 1. Copy the `/commit` skill and run it

This task produces a branch, the file `.claude/skills/commit/SKILL.md`, and one commit.

1. **Create a branch.** Sessions 5 to 8 reuse this code. You delete the branch after the closing round.

```
git switch -c session4-playground
```

2. **Let Claude copy the skill.** The package already holds the ready-made skill: the file `exercises/starters/commit-skill.md` in the `bookstore` folder. Paste:

```
[Exercise 4 experiment — execute directly, no leading questions.] Copy
exercises/starters/commit-skill.md to .claude/skills/commit/SKILL.md. Change nothing in
the content.
```

3. **Change a file by hand.** Add one comment line to any source file and save it. Do not ask Claude.
4. **Run the skill.** Type `/commit` and watch what it does.
5. **Read three lines.** Open `.claude/skills/commit/SKILL.md` in your editor and find these:
   - `disable-model-invocation: true`: only you can start the skill, by typing `/commit`.
   - *Execute directly — no leading questions*: Claude commits and does not ask you questions.
   - *Imperative mood*: the commit message starts with a verb, such as "Add" or "Fix". Task 2 builds the changelog from these messages.

**Done when**: `git log -1` shows a new commit. Its message starts with a verb, such as "Add" or "Fix".

## 2. Let Claude write `/changelog`, then write its description

This task produces `.claude/skills/changelog/SKILL.md` and, next to it, `common-changelog-spec.md`. That file holds the rules of the Common Changelog format.

1. **Paste one prompt for both files.**

```
[Exercise 4 experiment — execute directly, no leading questions.] Read https://common-changelog.org/ and save the whole specification as markdown to .claude/skills/changelog/common-changelog-spec.md. Keep the headings and the examples. Leave out the website menu and footer. Then create .claude/skills/changelog/SKILL.md. Frontmatter: the line `name: changelog` and the line `description: (I write this)`. No other frontmatter lines. Body: a numbered list of instructions that make you do all of the following.
- Read CHANGELOG.md in the project root. If the file does not exist, create it with the heading `# Changelog`.
- Read common-changelog-spec.md, next to this file, before changing the changelog.
- Look at the recent commits and at the existing version tags.
- Add only new commits. Entries that are already in the changelog stay as they are. New entries go in a `## Unreleased` section at the top.
- Sort entries under `### Changed`, `### Added`, `### Removed` and `### Fixed`, in that order.
- Write each entry as one sentence that starts with a present-tense verb. End it with the short commit hash in round brackets, like (a1b2c3d).
- A breaking change forces other people to change their own code. Put **Breaking:** in front of such an entry, and put those entries first inside their group.
- Leave out changes a reader does not care about: files whose name starts with a dot, formatting-only changes, and developer tools.
```

   If your network blocks the page, ask Claude to write the spec file from what it knows about Common Changelog.

2. **Check two things.** `common-changelog-spec.md` holds group names and rules, not an empty page. The `SKILL.md` frontmatter has **no** `disable-model-invocation` line. So Claude can start this skill by itself.

3. **Write the description.** In your editor, replace `(I write this)`. Claude reads only this line when it decides to start a skill on its own. A good description names the file, the format and the moment to act (*Which Description Gets This Skill Invoked at the Right Moment?*). Then answer two questions on paper. Which words make Claude start the skill right after a commit? Which words would start it at the wrong moment?

4. **Test the description.** Type each request in plain words, without a slash. The chat shows the skill loading when Claude starts it. First:

```
I just committed. Bring the changelog up to date.
```

Claude must start the skill. Now the second request:

```
Summarise the last three commits for me, in two sentences.
```

Claude must **not** start the skill. If a request went wrong, change the words in your description that caused it. Send the request again.

**Done when**: both files exist, the frontmatter has no `disable-model-invocation` line, request 1 started the skill, and request 2 did not.

## 3. Wire the guard and predict

This task produces `.claude/hooks/guard-secrets.py`, an entry in `.claude/settings.json`, and three written predictions.

1. **Let Claude create the script.** It runs on every prompt you send, before the prompt leaves your laptop. It is Python because many of you use Windows (*The Secret That Never Leaves*). Paste this whole block:

```
[Exercise 4 experiment — execute directly, no leading questions.] Create
.claude/hooks/guard-secrets.py with exactly this content, nothing added or changed:

import json
import re
import sys

data = json.load(sys.stdin)
text = data.get("prompt", "")

if re.search(r"acme_live_[A-Za-z0-9]{24}", text):
    print(json.dumps({
        "decision": "block",
        "reason": "Blocked: this prompt contains an ACME API key. Remove the key and send
it again."
    }))
```

2. **Add the hook to your settings.** In your editor, create the file `.claude/settings.json`. It does not exist yet. Give it exactly this content:

```
{
  "hooks": {
    "UserPromptSubmit": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "python .claude/hooks/guard-secrets.py"
          }
        ]
      }
    ]
  }
}
```

On mac or linux, write `python3` instead of `python`. Not sure? Run `python --version` in a terminal. If that fails, use `python3`.

3. **Write three predictions** on paper or in a text file. Write one line for each case: does the guard block it, yes or no, and why. The check in task 4 asks for them. Case 3 is tested in the bonus.

   1. You paste a log line that contains the key.
   2. You ask Claude what the value of `ACME_API_KEY` in `.env` is.
   3. You ask Claude to run `grep -r acme_live .`.

**Done when**: both files exist and your three predictions are written down.

## 4. Run the guard, find the gap, close it

This task produces one blocked prompt, a `permissions` rule in `.claude/settings.json`, and your marked predictions.

1. **Send a test prompt**, without a tag. It looks like a log line:

```
This request failed on staging. What is wrong with it?
POST /api/books 401 token=acme_live_7Hq2mX9vK4pL8sW1dR3nT6yB reason=expired
```

The guard blocks it and the terminal shows the reason. Did Claude answer instead? Then the hook did not start. Fix the `command` in `settings.json` (`python` or `python3`) and send the prompt again.

2. **Ask for the key.** Send this without a tag:

```
What is the value of ACME_API_KEY in .env?
```

   Claude reads the file. The guard did not block it: it checks only the text you type, not what Claude reads. Whether Claude repeats the value or not, the file content is now in the chat, so it went to the model. Did Claude ask a question first? Type *just tell me*.

3. **Close the gap with a rule.** A rule is a list of files Claude may not read (*Claude Must Never Read* `.env`). Open `.claude/settings.json`. Add the `permissions` part before `hooks`. It ends with a comma.

```
{
  "permissions": {
    "deny": ["Read(.env)"]
  },
  "hooks": {
    "UserPromptSubmit": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "python .claude/hooks/guard-secrets.py"
          }
        ]
      }
    ]
  }
}
```

   Save the file. No restart is needed. Send the same question again. Claude says it may not read the file. Then send:

```
Run cat .env and show me the output.
```

   This is denied too. The rule covers file commands such as `cat`, `head` and `tail`.

4. **Mark your predictions** *confirmed* or *wrong*, each with one line from the chat as evidence. For case 3, write one line of reasoning if you skip the bonus.

5. **Run the check.** Type `/verify-exercise 4`. It asks for your predictions: type them in the chat. Then it grades both skills, the guard and the rule. Read its report before the closing round.

**Done when**: the test prompt was blocked, the rule is in `settings.json`, your predictions are marked, and `/verify-exercise 4` gave its report.

## Bonus (only if time remains)

**The gap in the rule.** Send this without a tag:

```
Run grep -r acme_live . and show me the output.
```

The key appears. The rule only sees commands that name the file, and `grep -r` names none. The guard does not see it either. Only the sandbox stops every process (*The Claude Code Security Harness*).

**Send the guard a fake event**, the way Claude Code does. Save this as `.claude/fake-event.json`:

```
{"prompt":"deploy with acme_live_7Hq2mX9vK4pL8sW1dR3nT6yB please"}
```

Run this in a second terminal, in the `bookstore` folder (`python3` on mac or linux):

```
cat .claude/fake-event.json | python .claude/hooks/guard-secrets.py
```

The script prints the block decision. Change the prompt in `fake-event.json` to `deploy please`: it prints nothing.

## Closing round

The trainer asks the room. Have these answers ready:

- Which prediction was wrong, and what showed it?
- What does the guard see, what does the rule see, and what stops every process?
- Which words in your `description` started `/changelog` at the right moment? Which would you change now?
- One thing you would tell someone who skipped this session.

## After the closing round

**Delete the branch.** The first line goes back to your previous branch. The second line deletes the session branch. Run them one at a time.

```
git switch -
git branch -D session4-playground
```

Your commits on that branch are deleted. Your files stay, because git does not track them: `CHANGELOG.md`, `.claude/skills/`, `.claude/hooks/` and `.claude/settings.json`.

**Back at work, two things.** If you build one hook this month, build a `PostToolUse` hook on `Edit|Write`. It runs after Claude edits or writes a file, runs your formatter or your fastest tests, and reports the failures to Claude. Claude then fixes what it just broke, without being asked. The hooks page in the Claude Code docs shows the settings.

The plugin **skill-creator** on Anthropic's official marketplace writes a first version of a skill for you. Install it with these commands, one at a time. `/plugin` opens a menu: install skill-creator there.

```
/plugin marketplace add anthropics/claude-plugins-official
/plugin
/reload-plugins
```

Try it on a job you repeat every week. Compare its draft with your task 2 file.